

Neural Tangent Kernel, Lazy Training, and Feature Learning

English seminar study notes

These notes are an English companion to the seminar handout. The emphasis is on mathematical derivations that can be followed line by line, on separating theorem statements from interpretation, and on identifying the assumptions under which each claim is valid.

Primary readings

Jacot, Gabriel, Hongler, **Neural Tangent Kernel**, NeurIPS 2018.

Lee et al., **Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent**, NeurIPS 2019.

Chizat, Oyallon, Bach, **On Lazy Training in Differentiable Programming**, NeurIPS 2019.

Contents

1	Notation and the central question	2
1.1	Notation map	2
1.2	The question of the week	2
2	From parameter gradient flow to the NTK	3
2.1	Function-space dynamics	3
2.2	If the kernel is constant	3
2.3	General losses	3
3	Jacot et al.: the infinite-width NTK	5
3.1	Kernel at initialization	5
3.2	Kernel constancy during training	5
4	Linearization and kernel regression	6
4.1	First-order Taylor model	6
4.2	Minimum-norm solution	6
4.3	Lee et al.: wide networks follow their linearization	6
5	What “lazy training” means	7
5.1	Small parameter displacement	7
5.2	Scaling can force laziness	7
5.3	Lazy does not mean slow	7
6	Feature learning and the scaling dichotomy	8
6.1	Mean-field versus NTK scaling	8
6.2	Yang–Hu: feature learning in infinite-width limits	8
6.3	μP as a practical parameterization principle	8
7	How to compare the regimes	9
7.1	A compact table	9
7.2	Diagnostics in experiments	9
7.3	What NTK theory does not say	9
8	Presentation checklist	10
8.1	Eight equations to keep	10
8.2	Common mistakes	10
	Bibliography	11

1

Notation and the central question

1.1 Notation map

Let $f_\theta(x)$ be a neural network, $\theta \in \mathbb{R}^p$ its parameters, and $X = (x_1, \dots, x_n)$ the training inputs. Write

$$f_\theta(X) = [f_\theta(x_1) \quad \dots \quad f_\theta(x_n)]^\top, \quad J_\theta(X) = \frac{\partial f_\theta(X)}{\partial \theta}.$$

The empirical neural tangent kernel is

$$K_\theta(X, X) = J_\theta(X) J_\theta(X)^\top.$$

For square loss we use the residual $r_\theta = f_\theta(X) - y$.

Concept	Jacot et al.	Lee et al.	Chizat et al.
network	f_θ	$f(\theta, x)$	$h(w)$ / scaled model
tangent kernel	Θ_∞	$\hat{\Theta}_t$	tangent model
linearization	implicit in kernel limit	explicit Taylor model	lazy regime
feature learning	absent in strict NTK limit	deviation from linearization	non-lazy regime

1.2 The question of the week

Mean-field theory showed that infinite width can preserve nontrivial feature motion. NTK theory asks about a different limit: can a wide network remain so close to initialization that training is described by its first-order Taylor expansion?

The crucial distinction is

$$\text{width } m \rightarrow \infty \not\Rightarrow \text{a unique limiting dynamics.}$$

The parameterization, output scaling, learning rate, and initialization jointly decide whether the limit is kernel-like or feature-learning.

How to say it in the presentation The NTK is not introduced as an arbitrary similarity measure. It appears exactly when parameter gradient flow is pushed through the network Jacobian into function space.

2

From parameter gradient flow to the NTK

2.1 Function-space dynamics

For square loss

$$\mathcal{L}(\theta) = \frac{1}{2} \|f_\theta(X) - y\|^2 = \frac{1}{2} \|r_\theta\|^2,$$

parameter gradient flow is

$$\dot{\theta}_t = -\nabla_\theta \mathcal{L}(\theta_t).$$

By the chain rule,

$$\nabla_\theta \mathcal{L}(\theta_t) = J_t^\top r_t.$$

Therefore

$$\dot{\theta}_t = -J_t^\top r_t.$$

Now differentiate the predictions:

$$\begin{aligned} \frac{d}{dt} f_{\theta_t}(X) &= J_t \dot{\theta}_t \\ &= -J_t J_t^\top r_t \\ &= -K_t r_t. \end{aligned}$$

Since y is fixed,

$$\boxed{\dot{r}_t = -K_t r_t.}$$

This identity is exact for any differentiable network under parameter gradient flow.

2.2 If the kernel is constant

If $K_t \equiv K_0$, then

$$\dot{r}_t = -K_0 r_t,$$

so

$$\boxed{r_t = e^{-K_0 t} r_0.}$$

If $K_0 = Q\Lambda Q^\top$, each eigenmode decays as $e^{-\lambda_i t}$. Thus the kernel spectrum determines the optimization time scales.

2.3 General losses

For a loss $\ell(f(X), y)$,

$$\dot{f}_t(X) = -K_t \nabla_f \ell(f_t(X), y).$$

The NTK therefore acts as a data-dependent preconditioner in function space.

How to say it in the presentation First derive $\dot{f} = -K_t \nabla_f \ell$ before discussing infinite width. The infinite-width theorem is then simply the claim that, under a particular scaling, K_t converges to a deterministic kernel and barely changes during training.

3

Jacot et al.: the infinite-width NTK

3.1 Kernel at initialization

For fully connected networks with standard independent initialization, layerwise covariances converge to deterministic recursions. Schematically,

$$\Sigma^{(0)}(x, x') = \frac{1}{d} x^\top x',$$

and if (u, v) is jointly Gaussian with covariance determined by $\Sigma^{(\ell)}$,

$$\Sigma^{(\ell+1)}(x, x') = \mathbb{E}[\sigma(u)\sigma(v)].$$

The derivative covariance is

$$\dot{\Sigma}^{(\ell+1)}(x, x') = \mathbb{E}[\sigma'(u)\sigma'(v)].$$

For a standard fully connected parameterization, the limiting NTK obeys a recursion of the form

$$\Theta^{(\ell+1)} = \Sigma^{(\ell+1)} + \Theta^{(\ell)} \dot{\Sigma}^{(\ell+1)}.$$

The exact base case and scaling depend on the architecture convention.

3.2 Kernel constancy during training

In the NTK parameterization, sufficiently wide networks move only a small distance in parameter space over $O(1)$ training time. As width tends to infinity,

$$K_t \longrightarrow \Theta_\infty$$

uniformly over finite time intervals under the theorem assumptions. The training dynamics then becomes

$$\dot{f}_t(X) = -\Theta_\infty \nabla_f \ell(f_t(X), y).$$

For square loss this is a linear ODE.

Caution “The NTK is constant” is an infinite-width statement under a specific scaling. Finite networks generally have $K_t \neq K_0$, and the amount of kernel evolution is itself a useful diagnostic of feature learning.

How to say it in the presentation Jacot et al. identify a regime in which a nonlinear neural network has a nonlinear forward map but nearly linear training dynamics around initialization. The nonlinearity survives in the deterministic kernel Θ_∞ ; the learned features do not substantially evolve.

4

Linearization and kernel regression

4.1 First-order Taylor model

Linearize around initialization θ_0 :

$$f_{\text{lin}}(x; \theta) = f_{\theta_0}(x) + J_0(x)(\theta - \theta_0).$$

Training this linear model by gradient flow yields the same function-space equation with fixed kernel K_0 .

4.2 Minimum-norm solution

Suppose the linearized model interpolates the training set. Let $\Delta\theta = \theta - \theta_0$ and solve

$$J_0\Delta\theta = y - f_0(X).$$

Gradient flow from $\Delta\theta = 0$ selects the minimum Euclidean-norm solution

$$\Delta\theta_\star = J_0^\top (J_0 J_0^\top)^+ (y - f_0(X)).$$

For a test point x ,

$$\begin{aligned} f_\star(x) &= f_0(x) + J_0(x)\Delta\theta_\star \\ &= f_0(x) + K_0(x, X)K_0(X, X)^+(y - f_0(X)). \end{aligned}$$

This is the ridgeless kernel-regression predictor with the empirical NTK.

4.3 Lee et al.: wide networks follow their linearization

Lee et al. make the linearization statement explicit: under suitable parameterization, width, learning-rate, and time assumptions, the trained wide network remains close to the output of its first-order Taylor model. One consequence is that optimization and prediction can be approximated by a kernel method without solving the nonlinear parameter dynamics.

How to say it in the presentation The bridge from NTK dynamics to kernel regression is not a metaphor. The first-order Taylor model has features $J_0(x)$, and its Gram matrix is exactly $J_0 J_0^\top = K_0$.

5

What “lazy training” means

5.1 Small parameter displacement

A differentiable model $h(w)$ is lazy when training stays in a neighborhood where

$$h(w) \approx h(w_0) + Dh(w_0)(w - w_0)$$

is accurate over the relevant trajectory. A useful dimensionless indicator discussed by Chizat–Oyallon–Bach compares initial error, second derivatives, and first derivatives, schematically

$$\kappa_h(w_0) \sim \|h(w_0) - y^*\| \frac{\|D^2h(w_0)\|}{\|Dh(w_0)\|^2}.$$

A small value means that the model can fit the target using a displacement small enough that curvature of the parameter-to-output map remains negligible.

5.2 Scaling can force laziness

Consider a scaled model whose output sensitivity grows with a scale parameter α . Then an $O(1)$ change in output may require only an $O(1/\alpha)$ parameter displacement. As $\alpha \rightarrow \infty$, the nonlinear model can therefore be forced into its tangent approximation even if the architecture itself is unchanged.

5.3 Lazy does not mean slow

The adjective refers to feature motion, not necessarily to optimization speed. Kernel eigenvalues can be large, so a lazy model may fit rapidly while still learning almost no new representation.

How to say it in the presentation Lazy training is best defined geometrically: the trajectory is short enough that the tangent features barely change. It is not synonymous with “wide,” “small learning rate,” or “slow convergence” by themselves.

6

Feature learning and the scaling dichotomy

6.1 Mean-field versus NTK scaling

For a schematic two-layer network

$$f_m(x) = \alpha_m \sum_{i=1}^m a_i \sigma(w_i^\top x),$$

choices such as $\alpha_m \propto m^{-1/2}$ and $\alpha_m \propto m^{-1}$ lead to different balances between output magnitude and parameter motion. In the kernel-like scaling, a small displacement of many parameters produces an $O(1)$ output change. In the mean-field scaling, neurons can move $O(1)$ and the feature distribution evolves.

6.2 Yang–Hu: feature learning in infinite-width limits

Tensor-program analyses make the scaling question systematic. Under broad architecture classes and stable parameterizations, infinite-width limits can fall into kernel-like regimes or genuine feature-learning regimes. The point is not that every parameterization is exactly one of two textbook formulas, but that nontrivial feature learning requires balancing layerwise updates so internal representations change at $O(1)$ scale.

6.3 μ P as a practical parameterization principle

Maximal Update Parameterization (μ P) chooses width-dependent parameter and learning-rate scalings so that updates remain nontrivial and stable as width changes. It is designed to preserve feature-learning behavior rather than collapse to a fixed kernel.

Caution Do not read the kernel/feature-learning distinction as a statement that feature learning is always superior. Kernel limits are analytically clean and can perform very well. The theoretical distinction concerns what the representation is allowed to change, not a universal ranking of predictors.

How to say it in the presentation The important axis is not finite width versus infinite width. It is frozen tangent features versus evolving features. Both behaviors can survive a width limit, depending on the scaling.

7

How to compare the regimes

7.1 A compact table

	NTK / lazy	Mean-field	Finite feature learning
parameter motion	vanishing/small	$O(1)$ after rescaling	architecture dependent
kernel evolution	vanishes in limit	generally nontrivial	measurable
representation	effectively fixed	distribution evolves	evolves
limit equation	kernel ODE	transport PDE	nonlinear SGD/GD
analysis tool	RKHS, spectra	Wasserstein, PDE	nonlinear optimization

7.2 Diagnostics in experiments

Common empirical questions include:

- How much does $\|K_t - K_0\|$ change?
- How far do parameters move relative to initialization?
- Do hidden representations align with new task structure?
- Does a frozen-feature or kernel baseline match the trained network?

No single diagnostic is a universal definition, but together they distinguish the regimes.

7.3 What NTK theory does not say

NTK convergence does not prove that SGD finds the same solution at realistic finite width, that feature learning is absent in practical networks, or that a kernel predictor has the same out-of-distribution behavior as the finite network.

How to say it in the presentation The cleanest seminar message is: NTK theory is a controlled linearization theorem. It becomes powerful precisely because it turns training into a fixed-kernel dynamical system; its limitation is the same simplification—the learned representation is frozen to first order.

8

Presentation checklist

8.1 Eight equations to keep

$$\begin{aligned}
 \dot{\theta} &= -J^\top r, \\
 K_\theta &= JJ^\top, \\
 \dot{r} &= -K_t r, \\
 r_t &= e^{-K_0 t} r_0 \quad (K_t = K_0), \\
 f_{\text{lin}}(x) &= f_0(x) + J_0(x)(\theta - \theta_0), \\
 f_\star(x) &= f_0(x) + K_0(x, X)K_0(X, X)^+(y - f_0(X)), \\
 \Theta^{(\ell+1)} &= \Sigma^{(\ell+1)} + \Theta^{(\ell)}\dot{\Sigma}^{(\ell+1)}, \\
 \rho_t &\text{ evolves nontrivially in the mean-field regime.}
 \end{aligned}$$

8.2 Common mistakes

- Saying that the NTK identity $K = JJ^\top$ is itself an infinite-width result; it is finite-width and exact.
- Saying the kernel is always constant during neural-network training.
- Equating infinite width with a unique kernel limit.
- Confusing small parameter displacement with small function change.
- Treating a representational theorem as a claim about what SGD necessarily learns.

How to say it in the presentation Thirty-second conclusion: pushing parameter gradient flow through the Jacobian produces the NTK exactly. In one infinite-width scaling the kernel freezes and the network follows its linearization; in other scalings, including mean-field and μP -style regimes, features continue to move at leading order.

Bibliography

- Jacot, A., Gabriel, F., and Hongler, C. *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. NeurIPS, 2018.
- Lee, J. et al. *Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent*. NeurIPS, 2019.
- Chizat, L., Oyallon, E., and Bach, F. *On Lazy Training in Differentiable Programming*. NeurIPS, 2019.
- Yang, G. and Hu, E. J. *Tensor Programs IV: Feature Learning in Infinite-Width Neural Networks*. ICML, 2021.